

• Sec-A

1. Define null value?

Ans in SQL, a null value signifies the absence of data in a specific database field. It's distinct from an empty string, zero, or spaces, which all represent actual values. A null value can be used in any data type and indicates the absence of a value.

2. What is the used rename operation?

Ans The Rename operation in SQL is used to change the name of an existing database object, such as a table, column, index, or constraint.

3. Syntax: ALTER TABLE current-name RENAME TO New-name;

3. Define tuple variable?

Ans it refers to variable that represent a single row or word in relational database table.

4. What is the use of union and intersection operation?

Ans Union operation is use to combine results of two or more select queries into a single result set.

Intersection operation is use to retrieve rows that appear in both result sets of two select queries.

5. Enlist the database Language?

Ans The database language are categorized four different type based upon the various operations.

- (i) Data Definition Language (DDL)
- (ii) Data Manipulation Language (DML)
- (iii) Data Control Language (DCL)
- (iv) Transaction control Language (TCL)

• Sec-B ($3 \times 7 = 21$)

1. Define basic operation of relational algebra?

Ans Relational algebra is a formal system for manipulating relations, which are tables in a relational database. Basic operations of relational algebra include:

- (i) Selection (σ): Selects rows from a relation that satisfy a given function. Condition.
- (ii) Projection (π): Selects specific columns from a relation, discarding the rest.
- (iii) Union ($\cup$): Combines tuples from two relations into a single relation, removing duplicates.
- (iv) Intersection ($\cap$): Retrieves tuples that appear in both of two relations.
- (v) Set difference ($-$): Retrieves tuple from one relation that do not appear in another.
- (vi) Cartesian product ($\times$): Generates a new relation by combining every tuple of one relation with every tuple of another.

(vii) Join ($\bowtie$): Combines tuples from two relations based on a common attributes.

(viii) Rename (ρ): This operation allows to rename attributes.

2. Explain types of Relational Calculus?

Ans Relational Calculus is a formal language used to formulate queries on relational databases. There are two main types of relational Calculus.

(i) Tuple Relational Calculus (TRC):

- in TRC, queries are expressed as formula consisting of variables and quantifiers over tuples.
- it focuses on the existence of tuples that satisfy certain conditions rather than how to retrieve them.
- TRC uses existential quantifier ($\exists$) and universal quantifier ($\forall$) to specify conditions on tuples.

Eg: $\{t \mid \exists t \in R (t.age > 30)\}$

(ii) Domain Relational Calculus (DRC):

- DRC focuses on what data to retrieve rather than how to retrieve it.
- in DRC, queries are expressed as formula consisting of variables and quantifiers over attributes (domain).
- Target finding values that meet a certain condition.

Eg. $\{name \mid \exists age, name \in R (age > 30)\}$

3. Enlist types of triggers?

Ans Trigger is a statement that a system executes automatically when there is any modification to the database. There are mainly three types of triggers.

(i) Before Triggers (BEFORE INSERT, BEFORE UPDATE, BEFORE DELETE):

→ These triggers are fired before the triggering event (such as INSERT, UPDATE, or DELETE) occurs.

→ Commonly used for ensuring data integrity or applying default values.

- BEFORE INSERT: activated before data is inserted into the table.
- BEFORE UPDATE: activated before data in the table is modified.
- BEFORE DELETE: activated before data is deleted from the table.

(ii) After Triggers (AFTER INSERT, AFTER UPDATE, AFTER DELETE)

→ These triggers are fired after the triggering event has occurred and the modification to the data has been made.

- AFTER INSERT: activated after data is inserted into table.
- AFTER UPDATE: activated after data in table is modified.
- AFTER DELETE: activated after data is deleted from the table.

(iii) Instead of Triggers:

→ These triggers are fired instead of the triggering event.

→ They are commonly used with views in database to implement complex update operations on views that cannot be directly modified.

Sec-C (3x11=33)

I. Table-1: Customer

Customer name	Street	city
Saurabh	A7	Patiala
Mehak	B6	Jalandhar
Sumiti	D9	Ludhiana
Ria	A5	Patiala

Table-6: Depositor

Customer name	Account No.
Saurabh	1111
Mehak	1113
Sumiti	1114

Table-2: Branch

Branch name	Branch city
ABC	Patiala
DEF	Ludhiana
GHI	Jalandhar

Table-3: Account

Account No.	Branch name	Balance
1111	ABC	50000
1112	DEF	10000
1113	GHI	9000
1114	ABC	7000

Table-4: Loan

Loan No.	Branch name	Amount
L33	ABC	10000
L35	DEF	15000
L49	GHI	9000
L98	DEF	65000

Table-5: Borrower

Customer name	Loan no.
Saurabh	L33
Mehak	L49
Ria	L98

- Queries 1: Find the loan no., branch, amount of loan is greater than or equal to 10000 amount.

Ans:
 Select loan_no, Branch-name, Amount
 From Loan
 Where Amount $\geq$ 10000;

OR

PQL: $\pi_{\text{loan-number, Branch-name, Amount}} (\sigma_{\text{Amount} \geq 10000} (\text{Loan}))$

- Queries 2: Find the loan number for each loan of an amount greater or equal to 10000.

SOL:
 Select loan-number
 From Loan
 Where Amount $\geq$ 10000;

OR

PQL: $\pi_{\text{loan-number}} (\sigma_{\text{Amount} \geq 10000} (\text{Loan}))$

- Queries: 3 - Find the names of all customers who have a loan and an account at the bank.

SOL:
 Select DISTINCT Customer-name
 From Borrower
 Inner Join Depositor ON
 Borrower.Customer-name = Depositor.Customer-name

OR

REL:

$$\pi_{\text{Customer-name}} ((\text{Borrower} \bowtie \text{Depositor}) \bowtie \text{Account})$$

Q4

Find the names of all customers having a loan at the "ABC" branch.

SOL:

```

Select DISTINCT Customer-name
FROM Borrower
Where Loan-number IN (
    Select Loan-number
    FROM Loan
    Where Branch-name = 'ABC'
);

```

OR

REL:

$$\pi_{\text{Customer-name}} ((\sigma_{\text{Branch-name} = 'ABC'}(\text{Branch}) \bowtie \text{Account}) \bowtie \text{Borrower})$$

Q2. Given a relation Student (Roll, Name, class, Fees, Team) with the following tuples:

Roll	Name	Department	Fees	Team
1	Gikash	CSE	22000	A
2	Tosh	CSE	34000	A
3	Kevin	ECE	36000	C
4	Ben	ECE	56000	D

Select all the Student of department ECE whose fees is greater than equal to 10000 and belongs to team other than A.

Ans

Select *

FROM Student

Where Department = 'ECE'

AND Fees >= 10000

AND Team != 'A';

Q3. What are aggregate functions? And list the aggregate functions supported by SQL.

Ans

Aggregate function in SQL are functions that perform a calculation on a set of values and return a single value as a result. These functions are commonly used in conjunction with the SELECT statement to summarize data.

Here are some common aggregate function supported by SQL:

- (i) Count(*): Counts the total number of rows in a table or group, including rows with Null values.
- (ii) Sum(): Calculate the total sum of values in a numeric column.
- (iii) Avg(): Calculate the average of values in a column.
- (iv) MIN(): Returns the minimum value in a column.
- (v) Max(): Returns the maximum value in a column.
- (vi) Group-Concat: Concatenates values from multiple rows into a single string, optionally separated by a delimiter.

There are some aggregate function which is supported by SQL.

[Signature]